

Phase 1 Review Packet

Everything produced so far, how to verify it independently, what was and was not tested, and the decisions that need a sign-off before Phase 0 starts.

Branch **claude/create-app-repository-1cxsih**

Head **13cc87d**

2 commits

Build **green**

Lint **clean**

Automated tests **none**

What exists

Two deliverables, with **very different review weight**. Read this before allocating time — most of the code in the repository is deliberately temporary, and reviewing it carefully would be wasted effort.

BLUEPRINT — DECISIONS THAT ARE EXPENSIVE TO REVERSE

CODE

Suggested split of a ~2 hour review: ~1h45 blueprint · ~15 min code skim

#	DELIVERABLE	WHERE	SIZE	STATUS
D1	Phase 1 technical blueprint Stack, schema, ledger design, permissions, security, roadmap	commit 13cc87d docs/architecture/	~9 000 words 3 diagrams	NEEDS DEEP REVIEW Architecture only — no code written from it.
D2	Working prototype Mobile-first PWA directory: browse, search, detail, submit, saved, offline	commit 0af2da2 src/	2 243 LOC 9 routes	SKIM ONLY Runs and is verified, but ~70% is replaced in Phase 0.

WHY D2 IS NOT THE FOUNDATION

D2 was built to answer "can we create an app in this repository?" before the SaaS brief arrived. It has **no database, no authentication, no payments and no money** — listings live in an in-memory object that resets when the server restarts. What survives into Phase 0 is the UI layer (design tokens, PWA shell, mobile components) and two patterns worth keeping: URL-driven search state, and server-side revalidation of every submission. The data layer is discarded.

The blueprint is published as a browsable document here: [Afrinext Technical Blueprint](#). The same file is in the repository at `docs/architecture/blueprint.html`, with a Markdown decision index beside it at `docs/architecture/README.md` — that index is the fastest way in, and is the thing worth diffing as decisions change.

Verify it yourself

Nothing below needs credentials, an account, or network access beyond npm and GitHub. Roughly ten minutes end to end.

1

Check out the branch

```
git clone https://github.com/abdoulasani/Afrinext.git
cd Afrinext
git checkout claude/create-app-repository-1cxsih
git log --oneline main..HEAD # expect exactly 2 commits
```

2

Install

Node 22 or newer. No postinstall scripts, no native builds.

```
npm install
```

3

Typecheck, lint and build

`next build` runs the TypeScript check, so a green build is also a green typecheck. Both should exit zero with no warnings.

```
npm run lint
npm run build
```

Expected: **Compiled successfully**, **Finished TypeScript**, and a route table listing 9 routes.

4

Run it and exercise the API

```
npm run start & # serves the production build on :3000

# search, filter and sort
curl -s 'localhost:3000/api/listings?q=shea'
curl -s 'localhost:3000/api/listings?category=logistics&verified=1'
curl -s 'localhost:3000/api/listings?sort=newest'

# server-side validation – expect HTTP 422 with per-field errors
curl -i -X POST localhost:3000/api/listings \
  -H 'Content-Type: application/json' \
  -d '{"name":"X","tagline":"short","category":"nope","contact":{}}'

# malformed body – expect HTTP 400
curl -i -X POST localhost:3000/api/listings \
  -H 'Content-Type: application/json' -d 'not json'

# unknown listing – expect HTTP 404
curl -i localhost:3000/api/listings/does-not-exist
```

5 Open it on a phone-sized viewport

`localhost:3000` at 390×844. Check the bottom tab bar, the browse filters, a listing page, and the submit form. Bookmark a listing and confirm it appears under *Saved* — that state is `localStorage`, per device, by design.

6 Read the blueprint

Start with `docs/architecture/README.md` — thirteen decisions in a table, then the open questions. Follow into the full document only where a decision looks wrong.

What was tested, and what was not

Stated plainly so the review starts from an accurate baseline rather than an assumption.

Verified, against a running production build

CHECK	RESULT
Production build and TypeScript check	PASS 9 routes emitted
ESLint	PASS clean, zero warnings
Search, category/country filter, verified filter, three sort orders	PASS correct result sets
Invalid submission	PASS HTTP 422 with per-field errors
Valid submission	PASS HTTP 201, appears in search, renders its own page
Duplicate business name	PASS unique slug generated (...-2)
Malformed JSON body	PASS HTTP 400
Unknown listing id	PASS HTTP 404 on both API and page
Render at 390×844 (Chromium)	PASS no console errors, no horizontal overflow on any screen
Save → Saved round trip	PASS persists across navigation

NOT TESTED — ASSUME NOTHING HERE WORKS UNTIL PROVEN

- **There is no automated test suite.** Every result above came from driving the running app by hand. Nothing guards against regression. Vitest and Playwright are scheduled in Phase 0.
- **No browser other than Chromium,** and no real device — desktop Chromium at a phone viewport is not the same as a mid-range Android on a 3G connection.
- **No load, soak or concurrency testing.** The in-memory store is not concurrency-safe and was never intended to be.
- **No security review.** The prototype has no authentication, so every API route is open by design — including `POST /api/listings`, which anyone can call. Acceptable for a demo, disqualifying for anything public.
- **No accessibility audit** beyond semantic markup, focus states and labelled controls.
- **Nothing from the blueprint is implemented.** No database, no ledger, no auth, no payments. The document describes work not yet begun.

Review checklist — the blueprint

These are the questions worth an opinion. Each names the decision, why it was made, and the specific thing that would make it wrong. A "no" on any row is more valuable than a "looks fine" on all of them.

Money — the decisions that cannot be retrofitted

DECISION	QUESTION FOR YOU
Double-entry ledger, immutable entries, per-transaction balancing enforced by a deferred database trigger	Is the invariant set complete? Specifically: is a deferred <code>CONSTRAINT TRIGGER</code> the right enforcement point, or would you prefer the balance check inside a stored procedure that owns all posting?
Pending and available are two separate ledger accounts; settlement is a transfer between them	Agree, or would you model it as a status on the transaction? The claim is that two accounts make "why is my money still pending" answerable by pointing at a row.
Balances are a cache in <code>account_balances</code> , verified nightly against the sum of entries	Is nightly frequent enough? Should drift detection block payouts automatically, or only alarm?
Money is <code>bigint</code> minor units + currency code; the exponent comes from a <code>currencies</code> table because XOF and XAF have zero decimals	Any place in the plan where this leaks — API responses, CSV exports, the admin console?
Rounding: non-seller lines round half-up, the seller gets <code>gross - Σ(other lines)</code>	Does the residual-to-seller rule hold up commercially, and is it defensible if a seller queries it?
Commission rules resolved once at order creation and frozen in <code>order_fee_lines</code>	Is the resolution order (most-specific-wins, then priority, then <code>effective_from</code>) unambiguous enough to avoid two rules tying?

Stack — where I want a second opinion

DECISION	QUESTION FOR YOU
CONTESTED Drizzle ORM over Prisma	Your call, and it depends on the team. Drizzle for SQL-first migrations and native CHECK constraints; Prisma for migration tooling and DX on a 40-table schema. What does the team actually know?
pg-boss over BullMQ + Redis	The argument is transactional enqueue — a settlement job commits with the ledger rows it will act on. Do you accept the throughput ceiling that comes with it?
Domain logic in a framework-free <code>packages/core</code> , enforced by an ESLint boundary rule	Is the boundary drawn in the right place? Anything that will inevitably want to import from Next.js?
Database sessions rather than JWTs; phone OTP as the primary credential	Agree on revocability over statelessness? Any concern about session-table load at scale?
Postgres in <code>eu-west-3</code> (Paris) rather than <code>af-south-1</code>	UNVERIFIED This is an assumption about routing, not a measurement. Do you have real latency numbers from the launch cities?

Model and scope

DECISION	QUESTION FOR YOU
Narrow the first launch to digital products and courses, one country, one payment rail	~18 weeks instead of ~32. Does dropping physical goods and delivery from launch cost more commercially than the schedule saves?
Build the ledger in P1, before any product feature	Three weeks with nothing visible to show. Is that defensible to the business, and would you sequence it differently?
Referral is single-level, paid only from settled transactions, basis is platform revenue not gross	The bound is structural — the platform can never owe more than it earned. Does single-level cost too much growth to accept?
Roles are scoped rows; no <code>role</code> column on <code>users</code> ; IDOR prevented by scoping queries rather than post-fetch checks	Does the <code>authorize(actor, permission, scope)</code> choke point cover every case you can think of?
Paid video protected by short-TTL signed tokens and a viewer overlay, with DRM explicitly out of scope	Is that honest tier acceptable to tell instructors, or does the business need real DRM at launch?
iPay behind a provider interface, with <code>createPayout</code> optional; a manual transfer rail closes the economic cycle in P3	Anything in the interface that will not survive contact with the real API?

OUTSIDE ENGINEERING, BUT IT GATES THE SCHEDULE

The blueprint flags that holding and disbursing user funds may be a licensed activity under BCEAO rules in the UEMOA zone, with separate regimes in Nigeria and Ghana. That is a legal question, not an architectural one — but it gates Phase 6 (payouts) and could reshape the wallet's legal model. If you know the answer or know who does, that unblocks the largest open risk in the plan.

Review checklist — the code

Fifteen minutes. The question is not "is this production quality" — it is not, and it is not meant to be. The question is **whether the patterns worth keeping are the right ones**.

FILE	LINES	WHAT TO LOOK FOR
<code>src/lib/store.ts</code>	64	The storage boundary. Everything that reads or writes goes through here, so the module can be swapped for Drizzle without touching callers. Is the surface right? If it is wrong, it is wrong in Phase 0 too.
<code>src/lib/search.ts</code>	109	Search state lives entirely in URL params, so results are linkable and the back button works. Pattern carries forward; the implementation moves to SQL.
<code>src/lib/validate.ts</code>	104	Server always revalidates; client checks are a convenience. Becomes Zod schemas shared across HTTP, server actions and jobs.
<code>src/app/api/listings/route.ts</code>	32	Status-code discipline: 201 / 400 / 422 / 404. Note there is no authentication — deliberate for a demo, and exactly what Phase 0 fixes.
<code>src/lib/useSaved.ts</code>	74	<code>useSyncExternalStore</code> over <code>localStorage</code> , with a hydration guard so saved state never flashes on SSR. Replaced by server-side favourites once accounts exist.
<code>src/components/</code>	~700	Design tokens, mobile shell, safe-area handling. This is the part that survives into <code>packages/ui</code> .

Worth **not** spending time on: `src/lib/seed.ts` (310 lines of fixture data, deleted in Phase 0), the visual design (subject to change once there is a real product), and anything about how listings are ranked (the ranking function is a placeholder).

Sign-off sheet

Four items block Phase 0. Six can be settled during it. Nothing starts until the four are answered — an approval that skips them would be approving an unknown.

Blocking

#	QUESTION	MY RECOMMENDATION	WHO DECIDES
B1	Who is advising on whether Afrinext may hold and disburse user funds in the launch country?	Engage local counsel before Phase 6. Structure so funds sit with the licensed PSP and Afrinext records entitlement.	Business / legal
B2	Launch scope — digital and courses first, or physical goods at launch?	Digital and courses. ~18 weeks vs ~32, and it de-risks the hardest parts first.	Business
B3	Is the iPay contract signed, and can we get the documentation and sandbox credentials?	If not, the manual transfer rail from P3 becomes the launch payment method — plan for it openly rather than waiting.	Business
B4	Referral — single level only?	Yes. Tiers materially increase regulatory exposure and contradict the brief's own principle.	Business, with legal input from B1

Settle during Phase 0

#	QUESTION	DECIDES
S1	Team size and Prisma vs SQL experience	The ORM choice — the only reason it is still open
S2	Launch country and currency (Niger / XOF assumed)	Seed data, locale default, payment channels
S3	Hosting — managed (Vercel + Neon) or containers (Hetzner / Fly)	Deployment shape and unit cost
S4	Is the PWA sufficient, or is a store-distributed app required?	How much weight the REST layer carries early
S5	Cash on delivery at physical-goods launch?	Whether the rider cash-float model is in scope
S6	KYC — which documents are verifiable locally, and is there an existing provider?	Withdrawal tiers in Phase 6

RECORD THE VERDICT

Per item: APPROVED APPROVED WITH CHANGES REJECTED — and for anything other than a clean approval, the reason. I will revise the blueprint against the feedback and reissue it before any code is written, rather than starting and patching later.

What happens after approval



The gate is real, not ceremonial. Phase 0 writes the database schema and the permission model — both are load-bearing for everything after, and both change shape depending on the four blocking answers in section 6.

Phase 0 — foundations · 2 weeks

Turn the repository into the monorepo the blueprint describes, and stand up the parts every later phase depends on.

- pnpm workspaces + Turborepo; move today's app into `apps/web`, extract components into `packages/ui`.
- Postgres, Drizzle schema v1 (or Prisma, pending S1), forward and rollback migrations in CI.
- Authentication: phone OTP and email, database sessions.
- Scoped RBAC with the single `authorize()` choke point.
- i18n with French default; `packages/i18n` shared with the worker.
- CI: typecheck, lint, test, migrate up and down on every commit. Vitest and Playwright wired up — **this is where the missing test suite gets fixed.**
- Observability skeleton: structured logs with request and actor ids.

Exit criterion: a user registers by phone, signs in, switches language; CI is green on typecheck, lint, tests and migrations in both directions.

Phase 1 — the money spine · 3 weeks

Nothing visible ships. This is the part that cannot be retrofitted, so it is built before anything depends on it.

- `packages/core/money` — Money type, currency minor units, the rounding rule.
- `packages/core/ledger` — posting, immutability, the balance trigger, settlement holds.
- Commission engine with rule resolution and frozen fee snapshots.
- Idempotency keys on every money-moving operation.

- Nightly reconciliation job with drift alarms, and an admin ledger viewer.

Exit criterion: property-based tests over 10 000 randomised operations prove every transaction balances and the cached balance equals the sum of entries; an injected drift is caught by the reconciliation job. Your senior developer should be able to read that test file and agree it proves what it claims.

WHAT THE NEXT PACKET WILL CONTAIN

Phase 0 hands over a running monorepo, a migration you can run against your own Postgres, and a CI run to inspect. Phase 1 hands over the ledger with its property tests — and that one is the most important review in the whole project, because a ledger bug is discovered by users, in money.

8

Standing handoff format

Every phase from here produces a packet in this shape, so the review is the same shape each time and nothing depends on remembering what happened last month.

SECTION	CONTENTS
1 · What exists	Deliverables, commit hashes, size, and where to spend review time
2 · Verify it yourself	Copy-pasteable commands, no credentials needed, expected output stated
3 · Tested / not tested	Every check run and its result — and an explicit list of what was <i>not</i> covered
4 · Review checklist	Targeted questions, not "please review". Each names what would make the decision wrong
5 · Exit criterion	The phase's stated criterion, and the evidence that it passed
6 · Sign-off sheet	Blocking items separated from ones that can be settled in flight
7 · What happens next	Next phase scope and its exit criterion, so approval is informed

Two commitments that make the packet worth trusting: **nothing is reported as working unless it was run**, and anything unimplemented, unverified or assumed is labelled as such rather than left to inference. Where a check was skipped, the packet says so — see section 3 of this one, which lists six things that were not tested.

Prepared for review · branch `claude/create-app-repository-1cxsih` · head `13cc87d`